

OUPi Robot Sumo: Quick Start

Pololu Zumo 2040 kit (v3). Read this first.

All of the robot code is public on GitHub.

<https://github.com/behrouzefdari/oupi-zumo-sumo> has every robot program, the clean V1 baseline, the flashing scripts, and these docs. It is the V1 fork (by Behrouz Eftari) of the original project by John Hassell / OU (github.com/jhassell/oupi-zumo-sumo). Clone or download it to set up any laptop: `git clone https://github.com/behrouzefdari/oupi-zumo-sumo`

1. How the game works

Two self-driving robots fight in a ring made of **bare wood**. The boundary is the **physical edge of the board, a drop-off with no tape or line**. A robot wins by pushing the other one off the board. Each robot also watches the edge with its downward sensors and tries not to drive itself off.

Players don't steer the robots, and they don't change any code. Each student sets up a robot with its own buttons, choosing a Persona (Bear, Lion, or Wolf) and a team Color (Blue, Red, or Green). A facilitator uses **Claude Code** (on the laptop's command line) only to install or reset the robot program over the USB cable. A connected robot appears on the laptop as a writable drive, just like a thumb drive or SD card. Two students' robots then face off head-to-head in the ring, student against student, robot to robot, for a clear winner.

The robots find each other with infrared, including "hearing" the other robot across the ring; the moment one clearly sees an opponent it jumps to full power, and once it is pushing it stays committed. Each robot shows its Persona and Color on its main screen, and its team name and letter (A/B/C) during a match.

2. Set it up: the quick version

1. Get the kit from GitHub (clone the repo, or download the ZIP) and start **Claude Code** inside the `oupi-zumo-sumo` folder.
2. Plug a robot into the laptop (it appears as a USB drive named MicroPython).
3. Say **"restart"**: Claude flashes the approved V1 program and ejects the drive.
4. Reset the robot and calibrate if it's a new arena (**C** on the Main screen → wood → edge).
5. The student presses **B** (Settings) to choose a Persona and a Color, **C** (Back), then **A** to fight. The next student can change the settings the same way.

3. Set it up: step by step

Get the kit

- Clone the public repo (the repo *is* the kit): `git clone https://github.com/behrouzefdari/oupi-zumo-sumo`. No git? Open the repo page, click the green **Code** button, choose **Download ZIP**, and unzip it.
- Start **Claude Code** inside the resulting `oupi-zumo-sumo` folder so it loads the project rules. Each laptop just clones its own copy; the GitHub repo is the source of truth.
- `sumo2_BASELINE.py` is the approved V1 default (never edited). `custom/sumo2.py` is the working copy that gets flashed; in V1 it is identical to the baseline. Students never edit code.
- See which robots are connected and their A/B/C letter: `python3 scripts/identify_bots.py` (identify a robot by its letter, not the drive name).

The student workflow (buttons only, no laptop)

6. Power on or reset the robot. The Main screen shows its Persona and Color (defaults: Lion, Blue).
7. Use the three buttons on each screen. After a Persona or Color pick, the robot goes back to Settings:

Screen	A	B	C
Main	Start	Settings	Recalibrate
Settings	Persona	Color	Back
Persona	Bear	Lion	Wolf
Color	Blue	Red	Green

Students don't use a laptop, pick build options, or change code. Persona and Color can be changed only while the robot is stopped, and they return to Lion / Blue after a reset.

8. On the Main screen, press **A** to start (3-2-1 countdown). Press **A** again at any time to stop the robot.

9. If the arena or lighting changed, recalibrate with **C** on the Main screen first (see Calibration below).

There's no fixed "house" robot: students compete against each other, robot to robot. Each student sets up their own robot with its buttons, then send two into the ring.

Calibration (once per robot, per arena)

- Press **C** on the Main screen (when stopped). **WOOD SAMPLE:** sit the robot flat on the bare wood, press C. **EDGE SAMPLE:** hold its front sensors out over the board edge (the drop-off), press C.
- It saves the result and reuses it on every reset, so you calibrate once per arena. Press C on the Main screen anytime (when stopped) to redo it if you change boards or the lighting.

Buttons

Button	What it does
A	Start / stop a match
B	Open Settings (Persona / Color), only when stopped
C	Recalibrate the edge (Main screen), only when stopped

⚠ Most important rule: always EJECT

Always eject the MicroPython drive before you remove the USB cable or reset a robot. On the Linux desktop, right-click the MicroPython disk and choose *Eject* (or the file manager's eject button, or the flash script's --eject). Pulling the cable before ejecting can corrupt the robot's memory. If a drive shows 0 bytes or read-only, the robot is running a program, so press reset or replug it to remount, then flash.

If a robot won't respond

If a robot isn't reachable, doesn't mount, or shows an error on its screen, its operating system probably needs reloading. Copy the entire contents of bot_image/ (from the repo) onto the robot's MicroPython drive. That restores the MicroPython OS, the Pololu library, and every program. The sumo2.py in bot_image/ is an older snapshot without the Persona/Color menu, so then install the current V1 program (this also ejects the drive):
`python3 scripts/flash_robot.py custom/sumo2.py --eject`. Then reset the robot.

4. If your arena is PAINTED (or different)

Recalibrate first. This kit's arena is bare wood with a physical drop-off edge (no border). A painted board reflects the sensors differently, so press **C** on the Main screen (when stopped) and re-sample the new surface and its edge.

You may also need a code change. The edge detection currently learns the difference between **bare wood** and the **drop-off at the board's physical edge**. If the new arena breaks that (a painted surface, a **painted border line** instead of a drop-off, a glossy/reflective finish, a different color, or **no border at all**), calibration alone may not be enough. Describe the arena to Claude Code in plain words: the surface color and finish, and what the boundary is (a painted line and its color/width, just the physical edge, or none). Claude Code can then rewrite the edge-detection and calibration so the robots read the new arena correctly and stay in the ring.

All code & docs: <https://github.com/behrouzefdari/oupi-zumo-sumo> · Reset a robot to V1: say "restart" to Claude Code